Indian Institute of Technology Kharagpur

Department of Artificial Intelligence

Assignment 6: Seq2Seq Translation and Long-Term Dependency Modeling (LSTM & GRU)

Laboratory Instructions & Global Hyperparameters

Duration: 3 Hours.
Environment: A GPU-enabled environment is strictly required for Seq2Seq training and gradient tracking components.
Dataset Files Provided: `train.jsonl`, `val.jsonl`, and `test.jsonl` (English to Rovelic parallel sentences). Each line contains a JSON object with fields: `src`, `tgt`, `n_clauses`, and `length_bucket` (short, medium, long).
Reproducibility: Fix all random seeds to 42 (`torch.manual_seed(42)`, `np.random.seed(42)`, `random.seed(42)`).

Mandatory Architecture & Training Hyperparameters:

- `MAX_LEN = 50`
- `BATCH_SIZE = 64`
- `EMBEDDING_DIM = 64`
- `HIDDEN_DIM = 128`
- `LEARNING_RATE = 0.001`
- `EPOCHS = 10`
- `TEACHER_FORCING_RATIO = 0.5`
- `CLIP_MAX_NORM = 1.0`

Important Note: Every deliverable requires only code, a computed numerical value, or a visualization. Mathematical operations in Section 2 must be implemented purely in NumPy without utilizing PyTorch autograd.

1. Dataset Loading and Preparation

1.1. Task 1.1: Dataset Parsing *[2 Marks]*

Implement `load_jsonl(path)` to parse the English-to-Rovelic parallel dataset, extracting `src`, `tgt`, `n_clauses`, and `length_bucket` attributes for train, validation, and test sets.

1.2. Task 1.2: Sequence Processing *[2 Marks]*

Tokenize raw text via whitespace splitting. Wrap sequences with `<sos>` and `<eos>` tokens, truncate to `MAX_LEN = 50`, and pad shorter sequences with `<pad>`.

1.3. Task 1.3: Vocabulary Construction *[1 Marks]*

Build special token mappings (`<pad>: 0`, `<sos>: 1`, `<eos>: 2`, `<unk>: 3`) and create vocabulary dictionaries (`token_to_idx` and `idx_to_token`) using only the training split. Map unseen validation and test tokens to `<unk>`.

1.4. Task 1.4: Batch Generator *[2 Marks]*

Construct a mini-batch iterator function `get_batches(...)` yielding source tensors, target tensors (as `torch.long`), and length bucket metadata lists transferred to the target device.

2. The Math of Gated Cells (NumPy From Scratch)

2.1. Task 2.1: LSTM Cell Mechanics *[5 Marks]*

Implement a standalone NumPy function `lstm_cell_forward(x_t, h_prev, c_prev, W, b)` computing the internal gating operations from scratch:

$$
\begin{aligned}
f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) && \text{(Forget Gate)} \\
i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) && \text{(Input Gate)} \\
\tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c) && \text{(Candidate Cell State)} \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{(Updated Cell State)} \\
o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) && \text{(Output Gate)} \\
h_t &= o_t \odot \tanh(c_t) && \text{(Hidden State)}
\end{aligned}
$$

Given fixed dummy inputs ($x_t \in \mathbb{R}^4, h_{t-1} \in \mathbb{R}^4, c_{t-1} \in \mathbb{R}^4$), compute and print the numerical output vectors h_t and c_t.

2.2. Task 2.2: Theoretical Parameter Calculation *[5 Marks]*

Demonstrate an understanding of the cell's mathematical capacity by calculating the exact number of learnable parameters (weights and biases).

Write a Python function `calculate_parameters(input_dim, hidden_dim)` that analytically computes and returns the total parameter count for a standard RNN layer, a GRU layer, and an LSTM layer based on their internal equations. Print the parameter counts for all three architectures utilizing the assignment's global variables (`EMBEDDING_DIM = 64` and `HIDDEN_DIM = 128`).

3. Gated Seq2Seq Architectures (PyTorch)

3.1. Task 3.1: GRU vs. LSTM Encoder-Decoder Modules *[7 Marks]*

- **GRU Suite *[3 Marks]*:** Implement `EncoderGRU` and `DecoderGRU` utilizing `nn.GRU(..., batch_first=True)`. The final hidden state h_T serves as the context vector.

- **LSTM Suite** ***[4 Marks]*****:** Implement `EncoderLSTM` and `DecoderLSTM` utilizing `nn.LSTM(..., batch_first=True)`. Transfer both the hidden state h_T and cell state c_T tuples from encoder to decoder.

Both decoder modules must project hidden representations to target vocabulary logits via a linear projection head. Store the recurrent layer (`nn.GRU/nn.LSTM`) as an instance attribute on each encoder.

3.2. Task 3.2: Unified Gated Seq2Seq Wrapper *[5 Marks]*

Implement a unified `GatedSeq2Seq` wrapper module configurable for both `cell_type='gru'` and `cell_type='lstm'`. Implement step-by-step autoregressive decoding supporting dynamic teacher forcing controlled by `TEACHER_FORCING_RATIO = 0.5`.

4. Comparative Training & Exposure Bias

4.1. Task 4.1: Empirical Convergence & Efficiency Comparison *[5 Marks]*

Train both the GRU-Seq2Seq and LSTM-Seq2Seq models for 10 epochs using `CrossEntropyLoss(ignore_index=0)` and `Adam(lr=0.001)` with gradient clipping (`CLIP_MAX_NORM = 1.0`). Plot Training Loss vs. Epochs and Validation Loss vs. Epochs for both architectures on a shared canvas. Log and report total training time (seconds) and average inference latency per batch (milliseconds).

4.2. Task 4.2: Exposure Bias Analysis (Inference Mode) *[5 Marks]*

Models trained with partial teacher forcing often suffer from "exposure bias"—they learn to rely on ground-truth tokens and compound their errors when forced to generate purely autoregressively at test time. Using your already trained GRU-Seq2Seq model from Task 4.1, analyze this bias on the validation set.

1. Perform a forward pass over the entire validation set in pure autoregressive mode (`TEACHER_FORCING_RATIO = 0.0`) and compute the average Cross Entropy Loss.
2. Perform a second forward pass over the validation set in fully guided mode (`TEACHER_FORCING_RATIO = 1.0`, artificially supplying the ground-truth targets) and compute the average Cross Entropy Loss.

Print both validation loss values. The numerical gap between these losses quantifies the penalty the model pays when its crutch is removed.

5. Evaluating Long-Term Dependencies

5.1. Task 5.1: Length-Stratified BLEU Evaluation *[7 Marks]*

Evaluate both trained models (GRU-Seq2Seq and LSTM-Seq2Seq trained at a 0.5 forcing ratio) on the test split using greedy decoding. Compute smoothed Corpus BLEU scores independently across short, medium, and long sequence buckets. Present evaluation results in the following comparison table:

Model Architecture	Short Bucket	Medium Bucket	Long Bucket	Overall BLEU
GRU-Seq2Seq	-	-	-	-
LSTM-Seq2Seq	-	-	-	-

5.2. Task 5.2: Qualitative Translation Evaluation *[4 Marks]*

Perform greedy inference on the following 3 benchmark test sequences. Tabulate the source, ground truth reference, and predictions from both models side-by-side:

Short Sample (1 Clause):

- **Source:** *"the old girl chases the red ball"*
- **Ground Truth:** *"redai balex oldai girol chaun"*

Medium Sample (2 Clauses):

- **Source:** *"the old wizard throws the brave table and the blue fox finds the brave ball"*
- **Ground Truth:** *"braai balex bluai foxol finun yol braai tabex oldai wizol thrun"*

Long Sample (4 Clauses):

- **Source:** *"the old dog finds the blue book and the swift cat sees the blue mountain and the small robot follows the old lamp and the young dog greets the quiet cloud"*
- **Ground Truth:** *"quiai cloex youai dogol greun yol oldai lamex smaai robol folun yol bluai mouex swiai catol seeun yol bluai booex oldai dogol finun"*

Total: 50 Marks